Context Diagram

Figure 1: Context Diagram

Software Architecture Report – Freeplane

1. Introduction

Freeplane is a desktop application for mind mapping and knowledge organization. The analysis is based on the C4 model, which provides a hierarchical view of the system through context, container, and component diagrams. The goal is to describe the system structure, identify its main architectural elements, and evaluate its design in terms of modularity, extensibility, and maintainability.

---

2. Context Diagram

Diagram

Explanation

The context diagram presents a high-level view of the Freeplane system and its interaction with external actors and systems.

Freeplane is a desktop application used to create, edit, and manage mind maps. The primary actor is the **User**, who interacts with the system through a graphical interface to organize information visually.

In addition to standard users, the diagram includes an **Advanced User**, representing users who utilize advanced features such as scripting and plugins to automate tasks or customize behavior. This distinction highlights support for different expertise levels.

External Systems

- **File System**
  Stores and retrieves mind maps and related resources using file-based persistence.
- **Plugin Repository**
  A distribution platform where plugins are published and downloaded.
- **Scripting Engine**
  Enables execution of scripts for automation and customization.
- **Export Services**
  Converts mind maps into formats such as PDF or HTML.

Container Diagram

Figure 2: Container Diagram

- **Operating System**
  Provides runtime environment and system resources (file handling, GUI rendering).

Freeplane emphasizes: - Local data management
- Extensibility through plugins
- Flexibility via scripting

It does **not rely on cloud-based services**.

---

3. Container Diagram

Diagram

Explanation

The container diagram provides a more detailed view of Freeplane's internal structure and interactions.

Unlike web-based systems, Freeplane is a **desktop application**, meaning: - It is deployed as a **single unit** - It is internally **modular**

Main Container

- **Desktop Application (Java, Swing)**
  - Handles user interaction
  - Renders GUI
  - Coordinates system behavior
  - Acts as the **core component**

Supporting Containers

- **Persistence (File Storage)**
  - Saves and loads mind maps
  - Uses XML-based `.mm` files
  - Communicates with the File System
- **Plugin System**
  - Dynamically loads and manages plugins
  - Extends or modifies application behavior
  - Interacts with Plugin Repository
- **Scripting Engine**
  - Executes automation scripts
  - Used mainly by advanced users

- **Operating System**
  - Provides runtime environment
  - Handles memory, files, and rendering

Key Insight

The system follows a **modular monolithic architecture**: - Single deployment unit - Internally separated responsibilities - Improved maintainability and extensibility

---

4. Component Diagram(s)

Diagram(s)

(To be added)

Explanation

(To be completed)

---

5. Architectural Analysis

- **Clean Architecture Discussion**
- **SOLID Violations**
- **Quality Attributes**

(To be completed)

---

6. Conclusion

Summary

(To be completed)

- Strengths of the architecture
- Weaknesses and limitations